

LimbuOCR: An End-to-End Page-to-Book OCR System for Limbu, and a Pipeline Toward the Nepal Language Pack

Ashish Thapa Ampixa Labs ashish@ampixa.com

Abstract

Nepal’s languages are written in more than a dozen scripts (Limbu in Sirijonga, Newar in Prachalit, Santali in Ol Chiki, among others), yet almost none has optical character recognition good enough to turn a scanned newspaper or book into editable text, so their printed record stays trapped in page images. The obstacles are concrete: tiny training corpora, legacy fonts that store the wrong bytes, no rights-cleared benchmarks, and multi-column pages that defeat naive reading-order recovery. **LimbuOCR** is a deployed system that begins to close that gap for Limbu. It turns a scanned page, image batch, or PDF into Unicode text with page and line structure, reading order, and crops that lead back to every prediction; a script router and a 50-language registry extend the same path to further recognizers as their data and rights mature.

The Sirijonga recognizer is trained from 4,872 real-print line pairs across 16 newspaper editions, each crop and label taken from the same PDF and split by publication date. On the held-out silver test it reaches 0.474% codepoint character error rate (CER), 0.979% grapheme CER, and 85.92% exact-line accuracy; on complete held-out pages the full pipeline reaches 2.575% page CER with 96.85% line-detection F1.

Beyond Limbu, the same source-pairing workflow has produced real-print silver labels for Newar, Tirhuta, Ol Chiki, Sunuwar, Kirat Rai, and Gurung Khema (in several cases the first such data of any scale), staged training sets, a working Newar diagnostic recognizer, and two mechanism-level negative results that redirected those routes. These are engineering baselines against machine-converted text, not human-graded accuracy: native review, a wider independent page set, and genuine Devanagari-written Limbu evaluation remain open.

1. Introduction

The practical test for this project is straightforward. Someone should be able to hand LimbuOCR a scanned book, photographed pages, or a PDF and receive editable Unicode text. When the system is unsure, the editor should be able to find the source line and correct it without reconstructing the OCR run. Meeting that test requires much more than a low error rate on pre-cropped lines: the output has to retain page boundaries, reading order, line boxes, crops, confidence, script decisions, and source identity, and weak detections must remain visible for review.

LimbuOCR now covers that path. It accepts PDFs and

image batches, detects lines, routes Sirijonga and Devanagari crops, restores reading order, and writes editable and structured outputs. Reviewer decisions remain tied to the source crops, and only accepted corrections can enter a later training export. We evaluate the Sirijonga recognizer both on cropped lines and through the full page path, so detection and ordering errors remain part of the result.

One part of the data pipeline needs careful separation from the runtime system. Some newspaper PDFs display Sirijonga with Namdhinggo-family legacy fonts. Their stored character codes can be mapped to Unicode and aligned with line images rendered from the same page. This gives us scarce real-print training material, but it is a way of constructing evidence, not the inference method: the deployed recognizer starts from pixels and does not read a PDF text layer.

Limbu also previews the problems a broader OCR effort in Nepal will face. It appears in two visually different writing systems; available sources mix Unicode, legacy encodings, and cross-script transcription; and its newspaper pages add multiple columns and page furniture under low-resource constraints. We have parameterized the reusable parts—detection, routing, assembly, export, and review—through a 50-language registry and run the same source workflow across a wider newspaper archive. Section 10 reports what that produced and which artifacts are deliberately not yet used. The measured subject of this paper remains Limbu.

The paper makes five contributions:

1. **An operational Limbu book-OCR research prototype** connecting page/PDF intake, preparation, detection, script routing, recognition, reading order, editable export, and review in one auditable system.
2. **A real-print Sirijonga data path with provenance**, recovering 4,872 line pairs from 16 same-PDF page/text sources under a date-grouped, exact-label-leakage-filtered split.
3. **Line and page diagnostics on a date-disjoint silver test**, comparing ft-v2 with the previous deployment on 902 lines and through complete held-out pages—including the demonstration that page CER is meaningful only under a declared serialization: the same predictions score 41.34% or 4.329% depending on the reading-order contract.
4. **A testable boundary for two-writing-system support**: a source-pixel audit rejects a mislabeled Devanagari OCR result and specifies the evidence a genuine Devanagari-written Limbu benchmark

needs.

5. **A transferable language-pack pipeline:** the same pixel/text pairing, legacy-font conversion, and leakage-checked staging executed across the Nepal pack, yielding per-script real-print data, one diagnostic second-language recognizer, and two root-caused negative results, each reported with its current review status.

Four research questions organize the work: whether same-PDF pixels and legacy text layers can bootstrap Sirijonga recognition when transcription is scarce (RQ1); which page components dominate end-to-end error once line recognition is strong (RQ2); how a system should distinguish source script, language, transcription encoding, and support maturity (RQ3); and whether crop-linked review can reduce the time to a corrected book (RQ4). The experiments answer RQ1 and RQ2 in preliminary form, Section 8 answers RQ3 operationally, and RQ4 still needs a study with fluent reviewers.

2. Limbu OCR as a System Problem

Project sources use both Limbu and Yakthung for the language, and both Sirijonga and Devanagari for its written form; Unicode assigns the Limbu script U+1900–U+194F (Unicode Consortium 2025a). An OCR record therefore needs at least three separate fields: the **language** (Limbu), the **visual writing system** on the page (Sirijonga or Devanagari), and the **transcription encoding** of the desired output. A Sirijonga page paired with a Devanagari transcription is a valid cross-script transcription task, but it says nothing about recognizing Devanagari glyphs, and it changes the dictionary, recognition head, fonts, and error analysis. Historical PDFs add one more layer: a page may look like Sirijonga while its text layer stores Latin or private codes rendered through an embedded font. A known font map converts those codes into useful silver labels; the visible script is still Sirijonga, and the deployed model still reads pixels.

We describe support with four levels—**runtime** (a model loads and emits text), **operational** (the page/book workflow emits reviewable artifacts), **measured** (quality is evaluated on relevant real pixels), and **claim-grade** (frozen, rights-cleared, human-reviewed evaluation). Sirijonga is measured on date-disjoint silver diagnostics; the Devanagari route is merely operational; neither is claim-grade. This vocabulary keeps “supported” from doing too much work.

Two evaluation choices follow throughout: we score

machine-converted silver without calling it human ground truth, and we make source grouping, leakage checks, normalization, missing predictions, and reading order part of the metric definition rather than prose caveats.

3. Related Work

Tesseract remains the reference example of segmentation and recognition in a deployable engine (Smith 2007); LimbuOCR builds on PaddleOCR, whose PP-OCR family combines lightweight backbones, sequence modeling, and deployable recognition heads (Li et al. 2022; Cui et al. 2025). The recognizer measured here is the PP-OCRv5 mobile model (PPLCNetV3 backbone, SVTR neck, CTC/NRTR heads); CTC enables sequence recognition without character-level alignment (Graves et al. 2006) but does not solve layout, which is why we keep line CER, detection F1, matched-line recognition, and full-page CER separate. For low-resource scripts, fonts, standardized text, rights, and representative evaluation constrain progress at least as much as model size (Agarwal and Anastasopoulos 2024); synthetic rendering and glyph-centered contrastive learning broaden coverage (Sharma et al. 2024) without establishing real-page performance, which matches our ablation experience (Section 8.1).

We report NFC codepoint CER and extended-grapheme-cluster CER under the Unicode segmentation model (Unicode Consortium 2025b) because Limbu combining signs split perceived characters across codepoints. Datasets and models are treated as evidence objects in the sense of datasheets (Gebru et al. 2021) and model cards (Mitchell et al. 2019). Community-centered work on Yakthung digital heritage frames script support as part of community-controlled tooling (Limbu 2024); our focused search found no peer-reviewed public Sirijonga page-to-book benchmark with comparable reporting, so we describe ft-v2 as an internal silver baseline without a priority claim. The closest systems precedent is the historical-document platform tradition of Transkribus (Mühlberger et al. 2019) and postediting research on transparent, human-controlled suggestions (Poncelas et al. 2020).

4. The LimbuOCR Platform

LimbuOCR separates model development from book processing. The offline path turns audited sources and accepted corrections into datasets, models, and evaluation records; the runtime path turns page pixels into editable text and review tasks; human decisions, attached to their crops, connect the two.

The runtime accepts a line crop, a full page, an ordered image set, or a PDF rendered at a recorded resolution; capture tooling can deskew and perspective-correct photographs while retaining raw and prepared image hashes. PaddleOCR detects text on the full page, and each poly-

gon becomes a crop with page-relative geometry. The crop is the smallest reviewable evidence unit: its record links source page, prepared page, detector box, image, recognizer output, and final book line, so a missed line or wrong character can be traced to the stage that pro-

Evidence and learning path

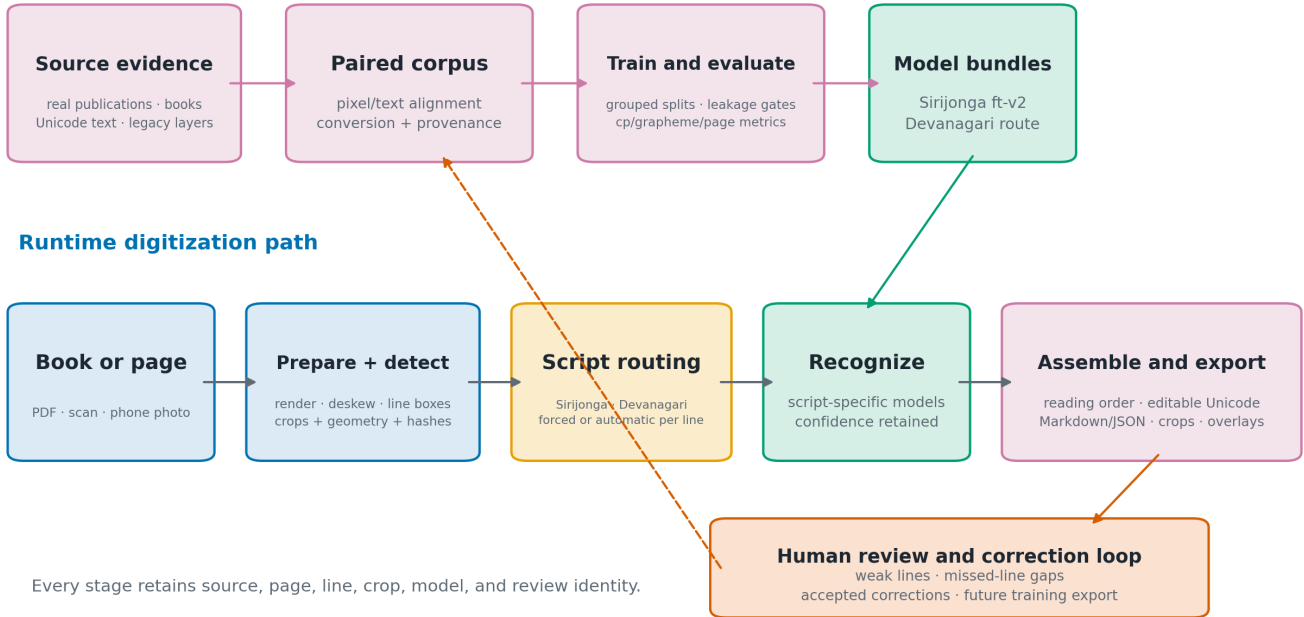


Figure 1: LimbuOCR system architecture. The upper path constructs and validates evidence and model bundles; the lower path performs page-to-book OCR. Review decisions remain attached to source crops and can feed later audited training exports.

duced it. Recognition may be fixed to Sirijonga or Devanagari or chosen per crop; the application bundles `limbu-sirijonga-ft-v2` (measured here) and `deva-v2`, a generic Devanagari recognizer that supplies a runtime route without any Limbu-specific accuracy result. Detected regions are assembled by row or column—the six-column newspaper test uses deterministic column-major order, recorded with every output and score—and the book path writes `book.txt`, `book.json`, annotated pages, crops, Markdown projections, and scorer-compatible document JSON. Weak lines, empty detections, suspected furniture, mixed scripts, and suspicious vertical gaps enter review queues; corrections fail closed against source and crop identity, and only explicitly accepted text can reach a training export.

The same components are parameterized beyond Limbu. A shared Unicode-block table identifies scripts, and a configuration-driven book runner resolves each language’s detector, recognizers, converters, and writing-system profiles from a 50-language pipeline registry. Registration does not grant permission to run: every recognizer is marked `champion`, `diagnostic-only`, `blocked`, or `failed-fit`, and an execution allowlist decides which statuses may enter the production path, so a withdrawn or legally blocked model stays visible but will not execute. Section 10 gives the evidence behind those statuses.

5. Building Real-Print Sirijonga Evidence

5.1 Source pairing and legacy-font recovery

The real-print material comes from Limbu pages in the *Naya Nepal* supplement of Gorkhapatra epaper PDFs. Both sides of every pair come from the same full PDF: we render the page image and extract the text layer from that same page, which prevents pairing a thumbnail with article text that was never laid out in it. Each retained row records the PDF and its hash, page number, rendered page, line geometry, raw codes, converted text, and crop. We manually checked the test page dated 2082-08-12: a six-column Sirijonga newspaper page whose rendered-image SHA-256 is recorded in the source-verification report. Publisher pixels are not reproduced here; the current owner determination covers derived counts and metrics, while redistribution awaits a formal license statement.

The pages display Sirijonga through Namdhinggo-family fonts, but their text layers are not Unicode Limbu. During extraction we group PDF spans into lines and convert the stored codes with `Limbu.map`, so the Unicode label and its image crop trace back to one PDF layout object. The pairing scales, but it is not gold ground truth: font maps can be wrong, spans can arrive out of order or vanish, and grouping and normalization can introduce errors. Every row is marked `pdf_text_layer_silver` and ex-

cluded from claim-grade evidence.

5.2 Grouped splitting and leakage control

The rebuilt collection contains 4,872 non-empty line pairs from 16 publication dates. Every line from a date stays in one split: after sorting the dates, indices congruent to 2 modulo 5 form the test split (2082-03-24, 2082-08-12, 2082-12-25), indices congruent to 0 form development, and the rest form training. Removing the 38 normalized labels that repeat across split boundaries leaves 4,834 lines: 2,711 training (nine dates), 1,221 development (four dates), and 902 test (three dates). Date grouping stops same-edition leakage and label filtering catches repeated boilerplate; near-duplicate phrases and shared page templates may remain.

5.3 Training mixture

The ft-v2 training bundle has 46,753 training rows and 1,221 development rows: 25,065 synthetic examples plus eight passes over the 2,711 real-print lines. Another 3,090 referenced synthetic images were lost in a storage failure and excluded rather than regenerated under the old names. Replay changes optimization weight, not sample independence. Training used an NVIDIA L4 for 40 epochs (about 8.5 GPU-hours), selecting epoch 39 on development; the bundle, YAML, log, checkpoint, export, predictions, and scores survive, and building this paper checks them without rerunning training.

6. Model and Evaluation Method

The recognizer is initialized from PaddleOCR’s pre-trained PP-OCRv5 mobile checkpoint: PPLCNetV3 backbone at scale 0.95, SVTR neck, CTC/NRTR heads, 48-by-320 inputs, at most 128 characters, trained 40 epochs with Adam, a cosine schedule from 10^{-4} , one

warm-up epoch, and batches of 64. The output dictionary remains the working `limbu-sirijonga-v2.txt`. The comparator is the preceding real-heavy Sirijonga export; both models run through one local CPU inference and scoring path on the same 902 images. ft-v2 never sees its test dates during training or selection; the older model’s history cannot be isolated to the same standard, so this is an engineering before/after comparison, not a controlled leaderboard.

Strings are normalized to NFC with whitespace preserved. CER is Levenshtein distance divided by reference length, micro-averaged over codepoints and over extended grapheme clusters; exact-line accuracy counts predictions matching the entire normalized reference. A page run starts from the complete rendered image: the detector proposes regions, the recognizer reads each crop, low-value outputs are filtered, boxes are ordered column-major, and a one-to-one matcher pairs predictions with references at IoU of at least 0.5. We report detection precision/recall/F1, CER over the fully assembled page text, CER and exact accuracy over matched lines, and missed and unmatched counts.

The 16-page regression set catches deployment regressions, but 13 of its dates occur in ft-v2 training or development, so we also rescore the exported documents for the three line-test dates (907 reference lines, date-disjoint from selection). We do not present page results as accuracy claims: there is no human review, only three independent pages against the project’s minimum of 30, no permission yet to redistribute pixels, and no complete multi-domain leakage audit. They are reported as reproducible diagnostics.

7. Results

7.1 Date-disjoint line recognition

Recognizer	Lines	Codepoint CER	Grapheme CER	Exact-line accuracy
Previous real-heavy export	902	1.379%	2.840%	63.75%
ft-v2	902	0.474%	0.979%	85.92%

The 902 test lines contain 34,814 codepoints and 16,760 grapheme clusters. ft-v2 cuts codepoint CER by 65.625% and grapheme CER by 65.55%; exact-line accuracy gains 22.17 percentage points. This measures agreement with the same-PDF silver layer, not with a fluent transcription. For RQ1, same-source pairing with date grouping,

real-line replay, and a fixed dictionary yields a useful internal baseline where transcription is scarce; only native review can say how much residual error—or apparent agreement—belongs to the silver labels.

7.2 Non-oracle pages on the held-out dates

System	Pages	Ref./pred./matched lines	Detection F1	Page cp CER	Page grapheme CER	Matched cp CER	Exact matched lines
Previous recog-nizer	3	907 / 903 / 877	96.91%	3.957%	6.280%	2.569%	45.95%
ft-v2	3	907 / 904 / 877	96.85%	2.575%	3.361%	1.012%	76.17%

The detector is fixed; most improvement is recognition. ft-v2 reduces assembled-page codepoint error by 34.94% and grapheme error by 46.49% relative; exact accuracy on the 877 matched lines rises 30.22 points. The run still misses 30 reference lines and leaves 27 predictions unmatched, so

these numbers include detection and assembly, not oracle crops.

7.3 Sixteen-page regression check

System	Pages	Page cp CER	Page grapheme CER	Detection F1	Matched cp CER	Exact matched lines
Previous recognizer, shared column-major	16	4.329%	6.714%	96.57%	2.657%	43.40%
ft-v2, shared column-major	16	2.722%	3.468%	96.50%	0.898%	79.27%

Most of these dates occur in training or development, so this table is an acceptance check for the bundled model, not an estimate on unseen books.

8. Lessons from the Experiments

8.1 Real-domain weighting beats indiscriminate synthesis

Synthetic data supplied script and glyph coverage, but more of it did not consistently help: one targeted synthetic/Unicode expansion worsened the frozen real-print score, and expanding the dictionary during a warm start replaced part of the recognition head and failed outright. ft-v2 worked after we kept the output space fixed and gave the aligned real-print lines much more replay weight. Synthesis should be treated as controlled coverage, ablated against a fixed real-domain test.

8.2 Page CER is defined by its serialization contract

Full-page CER compares two long strings, so it measures sequence as much as text, and the serialization rule is part of the metric definition. On the 16-page set, the same recognizer output scores 41.34% codepoint CER when reference and prediction are linearized under dif-

ferent rules—alongside 4,895 reference lines, 4,839 predictions, 4,700 geometric matches, 96.57% detection F1, and only 2.66% CER on matched lines. Applying one explicit column-major linearization to both sides resolves the contradiction: 4.329% codepoint and 6.714% grapheme CER over the identical predictions, with no model retrained and no recognized text altered. The mechanism is simple: on a six-column page, small vertical offsets make a global y-sort weave columns together, and edit distance counts the moved blocks as character errors. For RQ2, the catastrophic-looking number was mainly an inconsistent sequence definition; the remaining error belongs to recognition, missing and extra lines, crop quality, and layout generalization. Every page score now records its linearization, and the 41.34% and 4.329% figures must never be read as two OCR systems.

8.3 A target string does not identify the visual writing system

An early curriculum pack was labeled Devanagari-written Limbu because its JSON references contain Devanagari Unicode. The PDF pixels, on reinspection, show Sirijonga glyphs: the fonts include **YakthungBold** and **NamdinggoSILL**, and raw extraction yields ASCII-like legacy codes. On the inspected page the 14-line reference

has 408 Devanagari-block codepoints and no Limbu-block codepoints while the page itself is visibly Sirijonga—so the real task is Sirijonga pixels to Devanagari transcription, not Devanagari OCR. A surviving 31-crop diagnostic is therefore evidence about cross-script transcription, and its lost scoring bundle is a second reason to exclude it from the verified tables. Operationally (RQ3): manifests store `language`, `visual_script`, `source_encoding`, and `transcription_encoding` separately, `deva-v2` is called a generic runtime route, and a Limbu-specific Devanagari result will require visibly Devanagari pixels containing Limbu.

8.4 Review burden is the next system metric

None of the recognition scores says how long a book takes to correct. Low average CER can coexist with missing headings, confidently wrong lines, or an interface that forces the editor back to the source page for every error. The RQ4 study should measure lines routed to review, missing and extra lines, correction time per page and per thousand characters, accuracy after bounded review, and the share of a book needing resegmentation—review burden, rather than CER alone, decides production usefulness.

9. What Is Implemented

The repository contains a compact application (Line OCR, Paddle Page OCR, and Paddle Book OCR, plus

image-line fallbacks and diagnostic probes) and the larger experiment workflow: capture manifests, page and book outputs with source and crop hashes, triage and review queues, fail-closed correction application, and reviewed-only training export. A private CPU service runs the same application; superseded models and a failed-fit manuscript-Newa export remain in the bundle with model cards so their history stays inspectable. Model cards bind inference artifacts and dictionaries by SHA-256, and the paper build verifies those artifacts and code paths statically. What is *not* yet demonstrated: a complete human-reviewed book, a correction-time study, and any Limbu-specific Devanagari validation.

10. The Language Pack in Pipeline

Sections 5–8 report Limbu experiments. Here we record what the same workflow produced on the rest of the Nepal language pack: locate a source document, classify its text layer (Unicode, legacy glyph substitution, transcription, or none), build an explicit converter where the font is understood, pair text with pixels from the same PDF object, and check provenance and split leakage before staging a training set. Training and quality claims wait for rights and native-label review; most entries below have not cleared those gates.

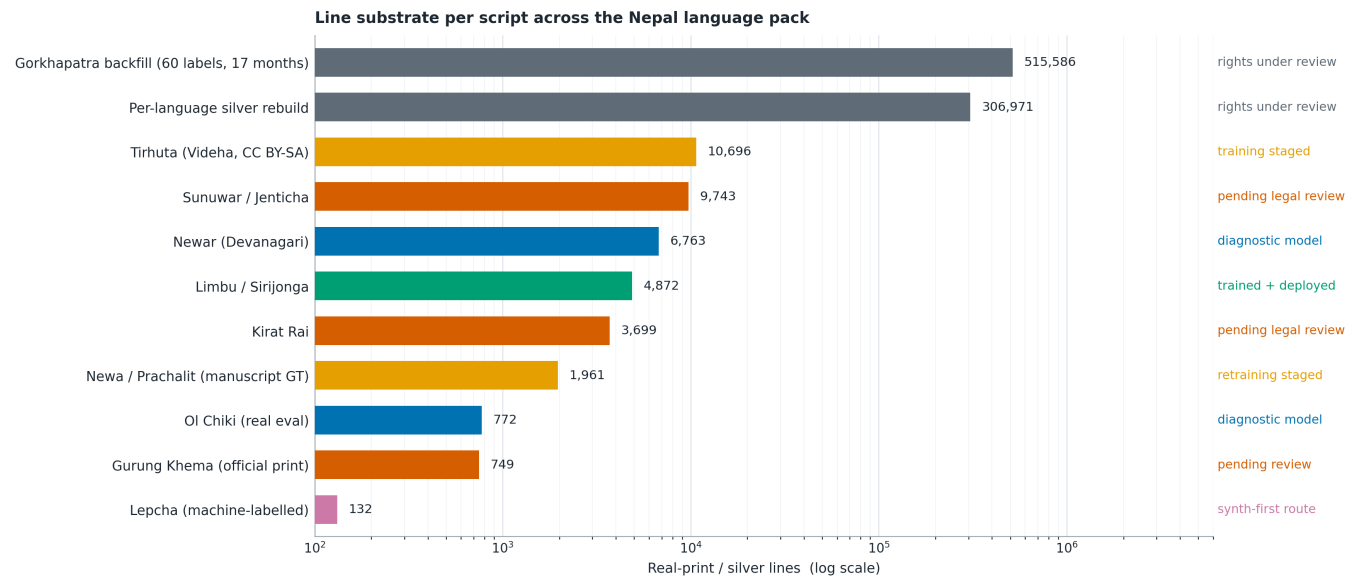


Figure 2: Real-print and silver line substrate per script, with pipeline status. Counts are read from the recorded pack artifacts at figure-build time. All rows are mechanical silver or diagnostic substrate, not human-reviewed ground truth.

10.1 Substrate at scale

Gorkhapatra’s *Naya Nepal* archive rotates pages for more than fifty languages. Applying the Limbu same-page rule at two depths retained **515,586** converted silver lines (monthly backfill, March 2024–July 2025) and **306,971** lines (per-language rebuild) across the 50-language registry; the collections overlap and are not deduplicated, so their sum is not a corpus size. These labels identify pages, not necessarily each line’s language—Nepali furniture appears on language pages—so line-level language identification must precede per-language training. Gorkhapatra is Nepal’s state-owned national daily; an owner determination (2026-07-22) permits the derived counts and metrics here, while page and crop redistribution awaits a formal license statement.

10.2 Newar, the second language

Newar (Nepal Bhasa) was chosen second because its Devanagari route could exercise the full Limbu workflow quickly. A publication-date split produced 4,096 training, 1,528 development, and 1,139 test lines; after a storage failure, the recovered labels lack the row-level provenance to reconstruct that isolation. On the frozen test, generic **deva-v2** (never trained on Newar) records 6.223% codepoint CER; warm-started **newar-ft-v1** records **4.655%** codepoint CER, 9.569% grapheme CER, and 24.50% exact-line accuracy. The Limbu page path gives 19.92% page codepoint CER, 91.16% detection F1, and 9.05% matched-line codepoint CER on 19 pages, pointing toward recognition and crop-domain error rather than reading order—though the evidence is too compromised for a firm attribution. A later check found label whitespace artifacts, 18 exact and 575 substring-level cross-split matches, missing provenance, and an open rights review, so the pack is not used for claims. The useful result is procedural: the shared path produced a working second-language diagnostic within days, and the review then stopped it from becoming an accuracy claim.

10.3 Scripts with staged training, pending review

Four other scripts have real-print material and staged training sets but no training run; archives are hash-pinned, development splits follow issue or page boundaries, and train/development intersections are zero.

Tirhuta. The Maithili e-zine *Videha* publishes a parallel Tirhuta edition (archived issues carry uploader-applied CC BY-SA 4.0 statements, recorded per issue), but its PDFs use legacy glyph-substitution fonts. Recovering that mapping, including a 149-entry conjunct extension, yielded **10,696** clean real lines—to our knowledge the first real-print Tirhuta OCR substrate at this scale. The staged split has 9,765 training and 844 development rows with a complete issue held out, plus a 48,960-image synthetic pretraining pack in an openly licensed font. No model is trained yet.

Ol Chiki. A magazine evaluation pack retains **772** clean lines (of 895 candidates; a dark-pixel guard removed 123 blank renders). The synthetic-trained v2 recognizer scores **8.26%** codepoint CER and 20.34% exact-line accuracy on it, but we withdrew that model from claim use after finding its training fonts came from the evaluation edition. A public-source-only v3 training set (5,307 images: 242 real train, 5,000 synthetic, 65 real development) awaits native review. Santali OCR exists elsewhere; this is pack coverage, not a first-ever claim.

Sunuwar and Kirat Rai. Government-hosted Sikkim Herald editions yielded **9,743** clean Sunuwar/Jenticha and **3,699** Kirat Rai lines through converters grounded in the relevant encoding proposals with no unresolved units. The splits (8,422/514 and 2,865/193) remain unused while legal review determines whether the State Portal permission covers model training.

10.4 Negative results that redirected routes

The Lepcha converter returned no lines from 18 Sikkim Herald editions because the article bodies are raster images with no live text layer—no font converter can ever recover them. The Lepcha path now starts from synthesis, anchored by a **132**-crop machine-labelled diagnostic that is explicitly ineligible for claims. The manuscript-Newa recognizer failed even on its training data for a structural reason: its mobile backbone pools every sequence to 40 CTC timesteps while 95.7% of manuscript lines exceed 40 characters, and CTC cannot emit more labels than timesteps, so no amount of data could fit. A T=200 configuration passes shape and loss checks, and a leakage-filtered 1,961-row training set with a sealed 635-line holdout awaits an approved run. Both failures identify a mechanism and changed the next experiment.

10.5 Source-first discipline for the hardest scripts

For Gurung Khema and native-script Newar, work stops at the source stage. Official Sikkim Herald Gurung Editions contain running Gurung Khema print, overturning our earlier belief that only synthetic material existed; a proposal-grounded converter can mechanically convert **749** of 759 candidate lines, but none is used pending provenance, native pixel-label review, and split checks. Newar itself is modeled as four writing-system profiles (Prachalit, Bhujimol, Ranjana, Devanagari): Bhujimol has no distinct Unicode encoding and Ranjana remains unencoded, so neither can reuse the Prachalit route without an explicit transcription policy. Bhujimol and Ranjana manuscript groups were catalogued and acquired only from institutional archives with explicit reuse statements, with every image hashed; there are still no reviewed labels, models, or metrics. In this registry, “supported” can mean only that input and output semantics are defined.

10.6 What the pipeline does not claim

No result in this section is claim-grade. The intended near-term system has one Sirijonga recognizer, one shared Devanagari recognizer, and per-script experts added only after rights-cleared, natively reviewed evaluation exists. What exists today is the auditable substrate—sources, converters, staged training sets, diagnostics, and explicit refusals to run unreviewed models—and each remaining model still has to earn its place per language, script, and domain slice.

11. Research Agenda

The next work closes specific gaps, in dependency order. **First, human-grounded Sirijonga evaluation:** fluent readers correct the three date-separated line and page sets (keeping silver and corrected text, with recorded disagreements), then the benchmark grows to at least 30 independent pages across newspapers and books, split by source; scoring the same predictions against silver and corrected references will show how much text-layer noise helps or hurts. **Second, genuine Devanagari-written Limbu:** candidate pages must visibly use Devanagari, contain Limbu, and carry references aligned to those pixels; `deva-v2` becomes the baseline and routing is tested only after both scripts have relevant page evidence. **Third, book and phone-capture robustness:** skew, curvature, uneven light, damaged print, varied fonts, and changing layouts, scored jointly with capture preparation, on at least one complete work. **Fourth, human-centered evaluation:** raw and post-review CER, missed and falsely confident lines, share of lines routed to review, correction time per page and per thousand characters, and reviewer agreement—queue policies compared at fixed final quality, with accepted corrections entering versioned training packs only after checks. **Fifth, structure and fallback:** tables, figures, and captions with structure-aware references; any large document model used for hard regions must report trigger rate, latency, and measured gain. **Sixth, gated transfer across the pack:** resolve the Sikkim Herald legal question, obtain native review per staged set, then run the approved chains (Tirhuta pretrain and fine-tune, Ol Chiki v3, Newa T=200) against frozen, edition-disjoint tests—one language, script, and domain slice at a time. This paper makes no claim-grade accuracy statement for any language, including Limbu.

12. Limitations and Rights

References and scope. The largest numerical limitation is reference quality: converted text layers carry encoding, ordering, grouping, and normalization errors, and a model trained on one publisher may learn its typography and PDF artifacts. Three held-out pages cannot represent other books, publishers, fonts, periods, or cameras. The previous deployment is an operational comparator,

not a baseline suite; external comparisons should wait for a frozen, human-reviewed image set. Only Sirijonga has a surviving Limbu-specific result—the presence of a Devanagari model is not evidence about Devanagari-written Limbu. Pack numbers add script-specific caveats: many silver lines are page-attributed rather than language-verified, the Newar pack has cross-split leakage and missing provenance, the Ol Chiki diagnostic was withdrawn over font overlap with its evaluation edition, and a pre-crash per-language Devanagari scorecard lost its backing artifacts and is omitted.

Rights. An owner determination (2026-07-22) for Gorkhapatra, Nepal’s state-owned national daily, permits the derived counts and metrics reported here but not redistribution of pages or crops; the paper uses artifact-derived diagrams only. Sikkim Herald training sets remain unused while the State Portal permission is reviewed for training use; only the Videha Tirhuta source has recorded per-issue CC BY-SA 4.0 statements. Any later release must state precisely which material its rights basis covers.

Community validation and reproducibility. Metrics cannot settle linguistic correctness, preferred script names, or community acceptance; fluent Limbu readers need a standing role in benchmark review, error taxonomy, and release decisions, with their corrections kept visible. The paper build regenerates figures and checks source counts, splits, metrics, model hashes, and runtime paths against local artifacts; it does not repeat the L4 training run, whose bundle, checkpoint, predictions, and scores remain available for inspection.

13. Conclusion

LimbuOCR is an operational research platform that turns page images and PDFs into editable, structured, reviewable Limbu text. The measured Sirijonga route records 0.474% codepoint CER, 0.979% grapheme CER, and 85.92% exact-line accuracy on a three-date silver test, and 2.575% codepoint CER with 96.85% detection F1 from complete pages. The same source and release workflow has produced large silver collections, converters, staged training sets, a Newar diagnostic, and two useful negative results across the Nepal language pack. Those artifacts mark a starting point: Sirijonga labels need native validation, Devanagari-written Limbu needs verified pages of its own, and every other language remains behind its rights and review gates. The serialization correction and the withdrawn curriculum result show why these boundaries belong in the method rather than a footnote. The next meaningful measure is review burden on a complete book—how much time the system saves while keeping missed text and uncertain lines visible.

Agarwal, Milind, and Antonios Anastasopoulos. 2024. “A Concise Survey of OCR for Low-Resource Languages.” In *Proceedings of the 4th Workshop on Nat-*

- ural Language Processing for Indigenous Languages of the Americas*, 88–102. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.americasnlp-1.10>.
- Cui, Cheng, Ting Sun, Manhui Lin, Tingquan Gao, Yubo Zhang, Jiaxuan Liu, Xueqing Wang, et al. 2025. “PaddleOCR 3.0 Technical Report.” *arXiv Preprint arXiv:2507.05595*. <https://arxiv.org/abs/2507.05595>.
- Gebru, Timnit, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. 2021. “Datasheets for Datasets.” *Communications of the ACM* 64 (12): 86–92. <https://doi.org/10.1145/3458723>.
- Graves, Alex, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber. 2006. “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks.” In *Proceedings of the 23rd International Conference on Machine Learning*, 369–76. Association for Computing Machinery. <https://doi.org/10.1145/1143844.1143891>.
- Li, Chenxia, Weiwei Liu, Ruoyu Guo, Xiaoting Yin, Kaitao Jiang, Yongkun Du, Yuning Du, et al. 2022. “PP-OCRv3: More Attempts for the Improvement of Ultra Lightweight OCR System.” *arXiv Preprint arXiv:2206.03001*. <https://arxiv.org/abs/2206.03001>.
- Limbu, Sumi. 2024. “Linguistic Heritage in Digital Landscape: A Collective Memory of Indigenous Script.” Master of Design thesis, University of Arkansas. <https://scholarworks.uark.edu/etd/5325/>.
- Mitchell, Margaret, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. 2019. “Model Cards for Model Reporting.” In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–29. Association for Computing Machinery. <https://doi.org/10.1145/3287560.3287596>.
- Mühlberger, Günter, Louise Seaward, Melissa Terras, Sofia Ares Oliveira, Vicente Bosch, Maximilian Bryan, Sebastian Colutto, et al. 2019. “Transforming Scholarship in the Archives Through Handwritten Text Recognition: Transkribus as a Case Study.” *Journal of Documentation* 75 (5): 954–76. <https://doi.org/10.1108/JD-07-2018-0114>.
- Poncelas, Alberto, Mohammad Aboomar, Jan Buts, James Hadley, and Andy Way. 2020. “A Tool for Facilitating OCR Postediting in Historical Documents.” In *Proceedings of LT4HALA 2020 – 1st Workshop on Language Technologies for Historical and Ancient Languages*, 47–51. European Language Resources Association (ELRA). <https://aclanthology.org/2020.lt4hala-1.7/>.
- Sharma, Prawaal, Poonam Goyal, Vidisha Sharma, and Navneet Goyal. 2024. “VOLTAGE: A Versatile Contrastive Learning Based OCR Methodology for Ultra Low-Resource Scripts Through Auto Glyph Feature Extraction.” In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics*, 881–99. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2024.eacl-long.53>.
- Smith, Ray. 2007. “An Overview of the Tesseract OCR Engine.” In *Proceedings of the Ninth International Conference on Document Analysis and Recognition*, 629–33. IEEE Computer Society. <https://research.google/pubs/an-overview-of-the-tesseract-ocr-engine/>.
- Unicode Consortium. 2025a. “The Unicode Standard, Version 17.0: Chapter 13, Limbu.” Unicode Standard. <https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-13/>.
- . 2025b. “Unicode Text Segmentation.” Unicode Standard Annex #29. <https://www.unicode.org/reports/tr29/>.